

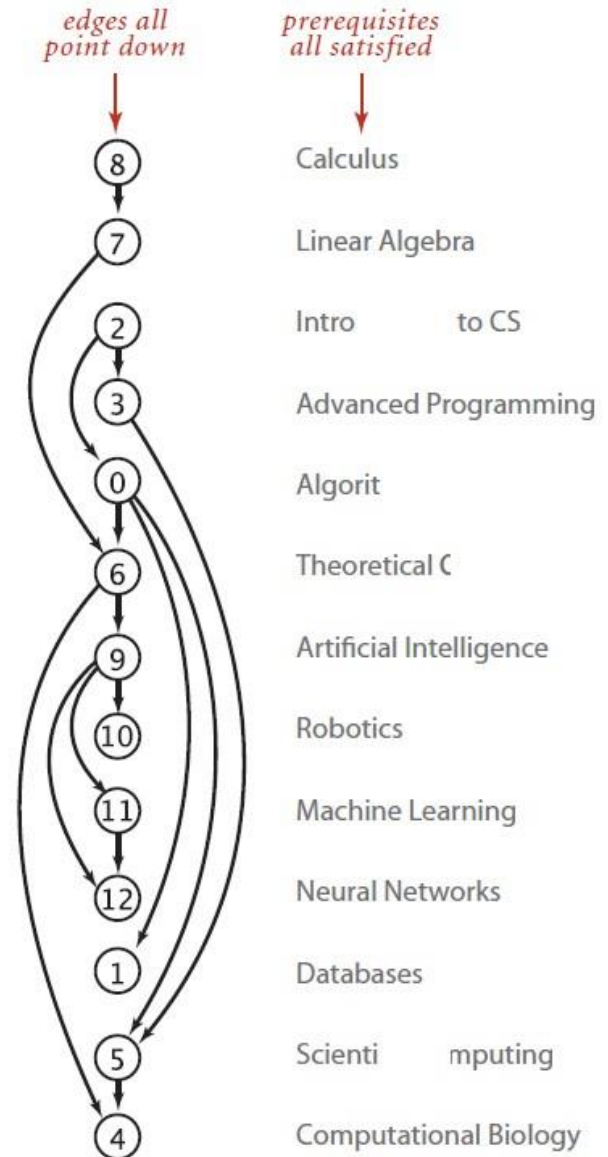
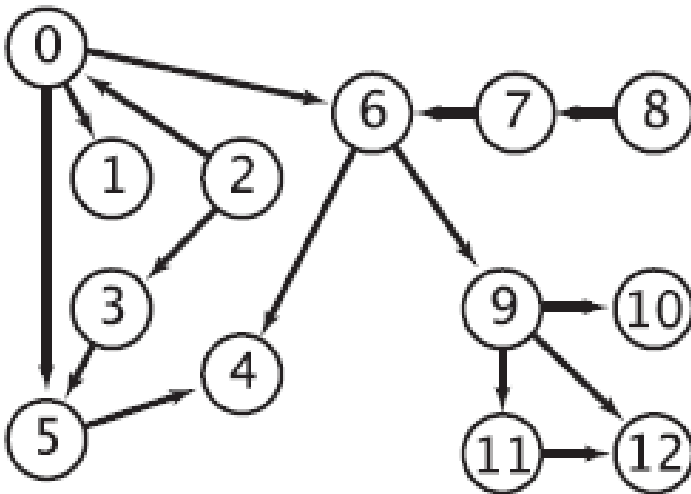


Componenti fortemente connesse in un grafo orientato

Giovedì 07 maggio 2026

Ordinamento topologico di un grafo orientato

- E' una sequenza dei vertici del grafo tale che tutti gli archi del grafo hanno un vertice di partenza che precede nella sequenza il vertice d'arrivo.



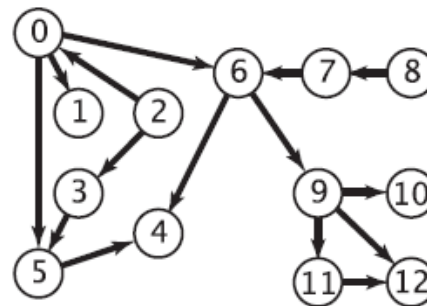
Topological sort

Ordinamento topologico e grafi aciclici

- **Teorema:** Un grafo orientato ha un ordine topologico se e solo se è un DAG.
- **Dimostrazione:** Se il grafo ha un ciclo orientato allora non ha un ordine topologico. Viceversa se il grafo è aciclico esiste un algoritmo che trova il suo ordine topologico.

Un algoritmo per l'ordinamento topologico

- In generale possiamo definire tre ordini diversi di visita DFS di un grafo orientato
 - *Preorder* : inserisci i vertici in una coda prima della chiamata ricorsiva
 - *Postorder* : inserisci i vertici in una coda dopo la chiamata ricorsiva
 - *Reverse postorder* : Inserisci i vertici in una pila dopo la chiamata ricorsiva.
- DepthFirstOrder.java trova gli ordini precentemente descritti
- Applichamole al grafo



DepthFirstOrder.class

```
public DepthFirstOrder(Digraph G)
{
    pre          = new Queue<Integer>();
    post         = new Queue<Integer>();
    reversePost  = new Stack<Integer>();
    marked       = new boolean[G.V()];

    for (int v = 0; v < G.V(); v++)
        if (!marked[v]) dfs(G, v);
}

private void dfs(Digraph G, int v)
{
    pre.enqueue(v);
    marked[v] = true;
    for (int w : G.adj(v))
        if (!marked[w])
            dfs(G, w);

    post.enqueue(v);
    reversePost.push(v);
}

public Iterable<Integer> pre()
{ return pre; }
public Iterable<Integer> post()
{ return post; }
public Iterable<Integer> reversePost()
{ return reversePost; }
```

preorder
is order of
dfs() calls



postorder
is order
in which
vertices
are done



	pre	post	reversePost
dfs(0)	0		
dfs(5)	0 5		
dfs(4)	0 5 4		
4 done		4	4
5 done		4 5	5 4
dfs(1)	0 5 4 1		
1 done		4 5 1	1 5 4
dfs(6)	0 5 4 1 6		
dfs(9)	0 5 4 1 6 9		
dfs(11)	0 5 4 1 6 9 11		
dfs(12)	0 5 4 1 6 9 11 12		
12 done		4 5 1 12	12 1 5 4
11 done		4 5 1 12 11	11 12 1 5 4
dfs(10)	0 5 4 1 6 9 11 12 10		
10 done		4 5 1 12 11 10	10 11 12 1 5 4
check 12			
9 done		4 5 1 12 11 10 9	9 10 11 12 1 5 4
check 4			
6 done		4 5 1 12 11 10 9 6	6 9 10 11 12 1 5 4
0 done		4 5 1 12 11 10 9 6 0	0 6 9 10 11 12 1 5 4
check 1			
dfs(2)	0 5 4 1 6 9 11 12 10 2		
check 0			
dfs(3)	0 5 4 1 6 9 11 12 10 2 3		
check 5			
3 done		4 5 1 12 11 10 9 6 0 3	3 0 6 9 10 11 12 1 5 4
2 done			
check 3		4 5 1 12 11 10 9 6 0 3 2	2 3 0 6 9 10 11 12 1 5 4
check 4			
check 5			
check 6			
dfs(7)	0 5 4 1 6 9 11 12 10 2 3 7		
check 6			
7 done		4 5 1 12 11 10 9 6 0 3 2 7	7 2 3 0 6 9 10 11 12 1 5 4
dfs(8)	0 5 4 1 6 9 11 12 10 2 3 7 8		
check 7			
8 done		4 5 1 12 11 10 9 6 0 3 2 7 8	8 7 2 3 0 6 9 10 11 12 1 5 4
check 9			
check 10			
check 11			
check 12			

queue



queue



stack



reverse
postorder



Ordinamento topologico

- Teorema: Il Reverse postorder in un DAG è un ordinamento topologico
- Dimostrazione: Consideriamo un arco $v \rightarrow w$. Quando $\text{dfs}(v)$ è chiamata, uno dei seguenti tre casi si verifica:
 - $\text{dfs}(w)$ è stata già chiamata e la sua esecuzione è terminata;
 - $\text{dfs}(w)$ non è ancora stata chiamata (w è unmarked), quindi $v \rightarrow w$ provocherà la chiamata $\text{dfs}(w)$ (e return), prima del return di $\text{dfs}(v)$;
 - $\text{dfs}(w)$ è stata già chiamata ma non ha terminato la sua esecuzione.Quest'ultimo caso è impossibile poichè implicherebbe l'esistenza di un cammino da w a v , cosa che contraddirebbe il fatto di essere in un DAG. Negli altri due casi possibili, $\text{dfs}(w)$ si concluderà prima di $\text{dfs}(v)$, pertanto w appare prima di v nella vista in postordine e dopo v nel reverse postorder. Quindi l'arco $v \rightarrow w$ avrà un vertice di partenza che precede nell'ordine il vertice d'arrivo, come desiderato.

Testare se un grafo orientato è aciclico

- Rilevamento di eventuali cicli:
 - esegui DFS, tenendo traccia di quali nodi stanno nello stack della ricorsione;
 - in presenza di un arco $v \rightarrow w$, verifica se w è nello stack.
- L'ordinamento topologico può essere eseguito in tempo $O(V+E)$

Implementazione

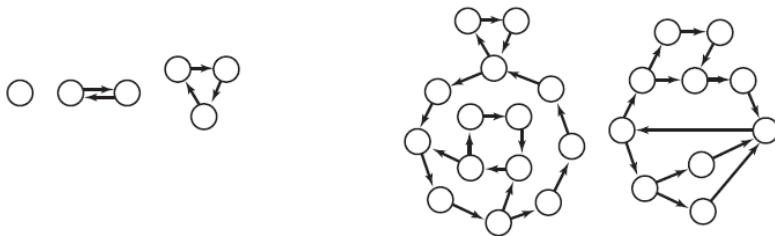
```
public class DirectedCycle {
    private boolean[] marked;           // marked[v] = has vertex v been marked?
    private int[] edgeTo;               // edgeTo[v] = previous vertex on path to v
    private boolean[] onStack;         // onStack[v] = is vertex on the stack?
    private Stack<Integer> cycle;      // directed cycle (or null if no such cycle)

    public DirectedCycle(Digraph G) {
        marked = new boolean[G.V()];
        onStack = new boolean[G.V()];
        edgeTo = new int[G.V()];
        for (int v = 0; v < G.V(); v++)
            if (!marked[v] && cycle == null) dfs(G, v);
    }

    private void dfs(Digraph G, int v) {
        onStack[v] = true;
        marked[v] = true;
        for (int w : G.adj(v)) {
            if (cycle != null) return;
            else if (!marked[w]) {
                edgeTo[w] = v; dfs(G, w); }
            else if (onStack[w]) {
                cycle = new Stack<Integer>();
                for (int x = v; x != w; x = edgeTo[x])
                    cycle.push(x);
                cycle.push(w);
                cycle.push(v);
            }
        }
        onStack[v] = false;
    }
    ...
}
```

Componenti fortemente connesse

- Ricordiamo che due vertici v e w sono fortemente connessi (strongly connected) se esiste un cammino orientato da v a w e se esiste un cammino orientato da w a v .
- La connettività forte è una relazione di equivalenza (e quindi i vertici sono partizionati in insiemi disgiunti).
- Una componente forte (strong component) di un grafo è un sottoinsieme massimale di vertici fortemente connessi.
- Due vertici sono fortemente connessi se e solo se esiste un ciclo orientato che li contiene entrambi
- Quante componenti fortemente connesse ha un DAG?



Individuare le componenti fortemente connesse in un Digraph

```
public class SCC
```

```
    SCC(Digraph G)
```

```
    boolean stronglyConnected(int v, int w)
```

```
    int count()
```

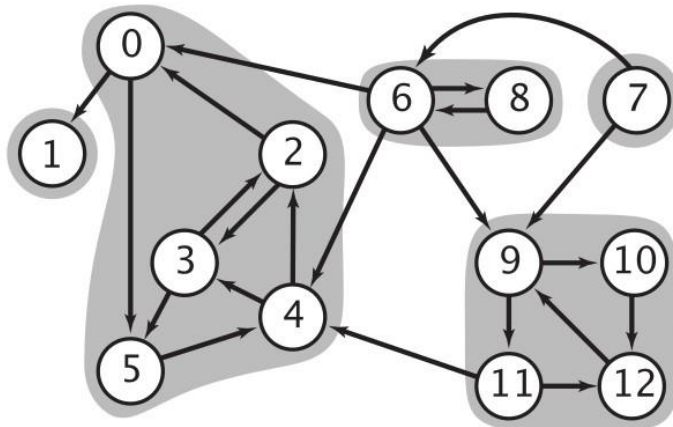
```
    int id(int v)
```

preprocessing constructor

are v and w strongly connected?

number of strong components

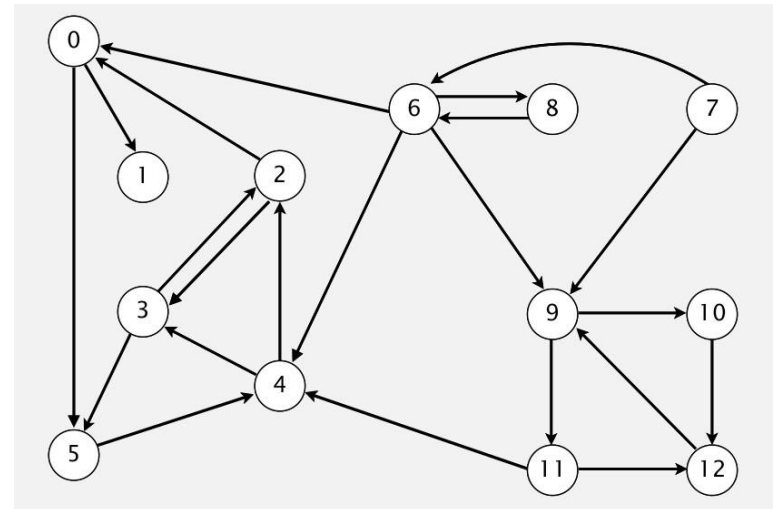
*component identifier for v
(between 0 and count()-1)*



Nota che un grafo G e il suo reverse G_R hanno le stesse componenti fortemente connesse

Algoritmo di Kosaraju-Sharir

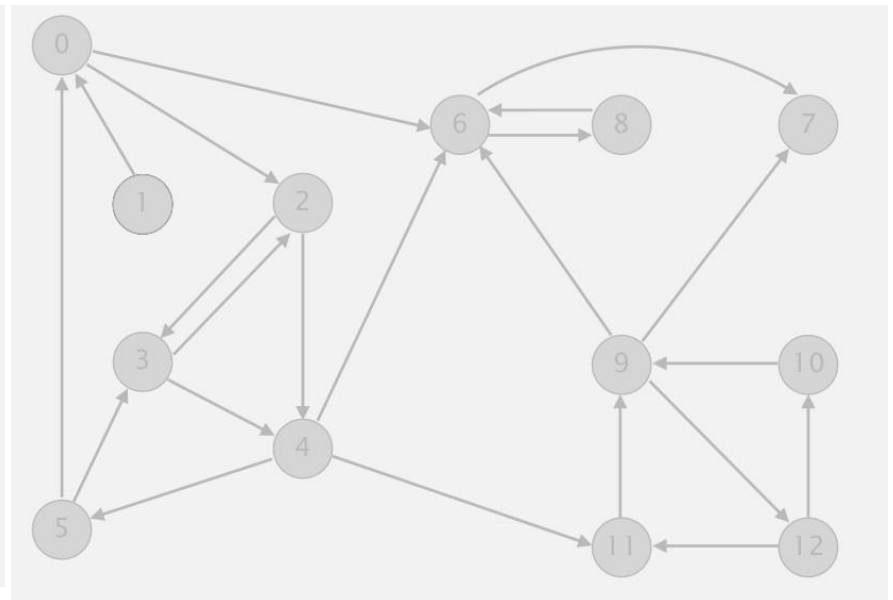
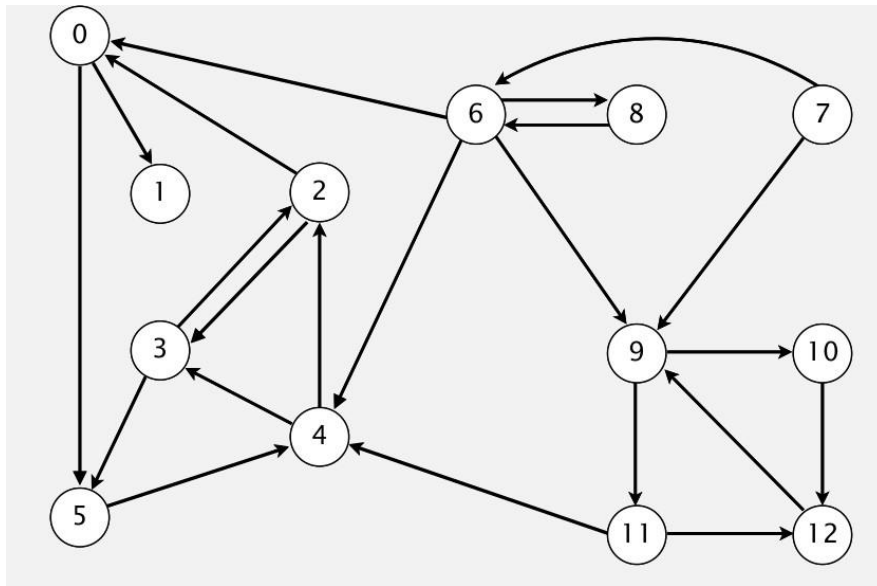
1. Calcola l'ordinamento reverse postorder su G_R ;
2. Esegui DFS su G , visitando i vertici non marcati seguendo l'ordinamento trovato nella fase 1.



Adiacenze di G :

0: 1 5; 1: 2 0; 2: 3 0; 3: 2 5; 4: 2 3; 5: 4; 6: 0 8 4 9; 7: 6 9; 8: 6; 9: 10 11;
10: 12; 11: 12 4; 12: 9

Fase 1: il grafo reverse



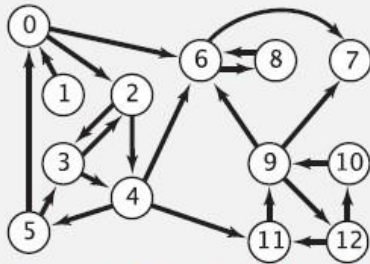
0: 6 2; 1: 0; 2: 4 3; 3: 4 2; 4: 11 6 5; 5: 3 0; 6: 8 7; 7: ; 8: 6; 9: 12 7 6;

10: 9; 11: 9; 12: 11 10

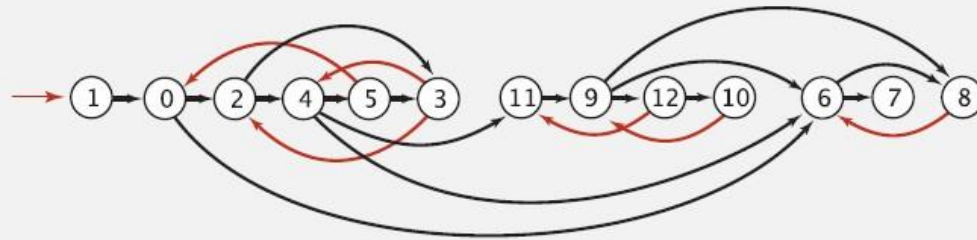
Reverse PostOrder ottenuto: 1 0 2 4 5 3 11 9 12 10 6 7 8

Fase 1

DFS in reverse digraph G^R



check unmarked vertices in the order
0 1 2 3 4 5 6 7 8 9 10 11 12



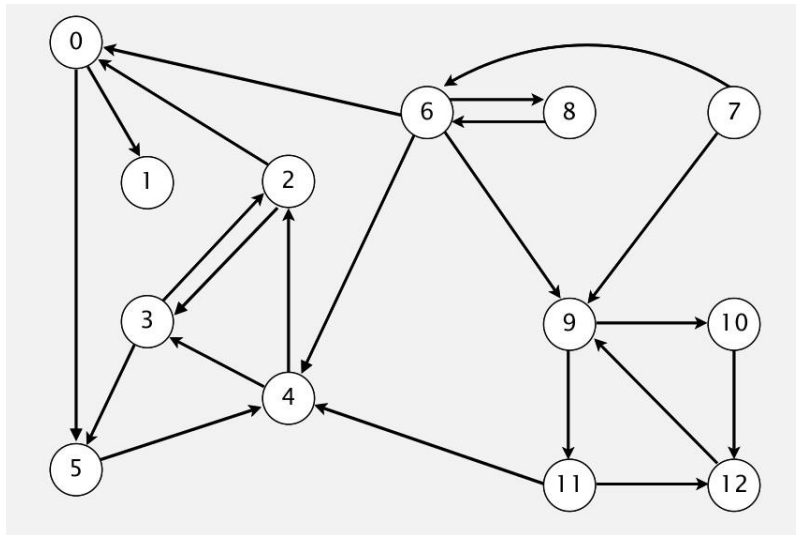
reverse postorder for use in second dfs()
1 0 2 4 5 3 11 9 12 10 6 7 8

```
dfs(0)
|
dfs(6)
|
dfs(8)
| check 6
8 done
dfs(7)
|
7 done
6 done
dfs(2)
|
dfs(4)
|
dfs(11)
|
dfs(9)
|
dfs(12)
| check 11
dfs(10)
| check 9
10 done
12 done
check 7
check 6
...

```

Fase 2

□
G



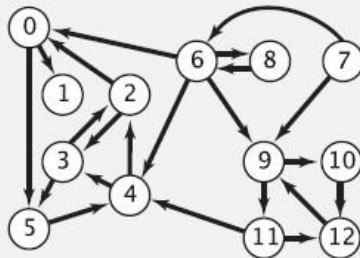
□ 0: 1 5; 1: 2 3 0; 2: 3 0; 3: 2 5; 4: 2 3; 5: 4; 6: 0 8 4 9; 7: 6 9;
8: 6; 9: 10 11; 10: 12; 11: 12 4; 12: 9

□ DFS da 1 0 2 4 5 3 11 9 12 10 6 7 8, contando il numero di componenti fortemente connesse.

v	id[]
0	1
1	0
2	1
3	1
4	1
5	1
6	3
7	4
8	3
9	2
10	2
11	2
12	2

Fase 2

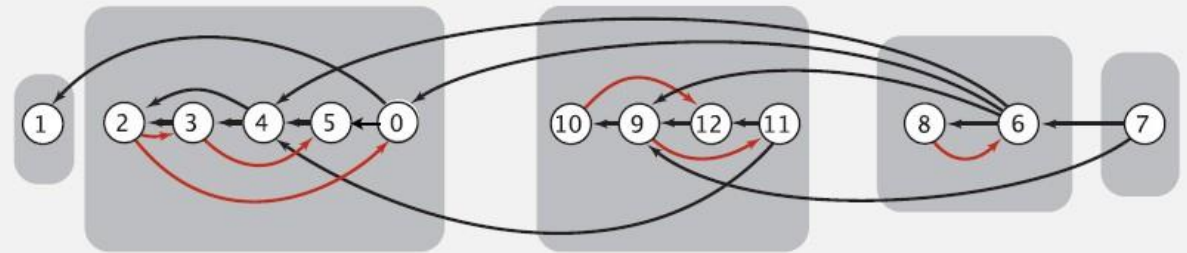
DFS in original digraph G



check unmarked vertices in the order

1 0 2 4 5 3 11 9 12 10 6 7 8

↑↑ ↑ ↑↑↑



dfs(1)
1 done

```

dfs(0)
  dfs(5)
    dfs(4)
      dfs(3)
        check 5
      dfs(2)
        check 0
        check 3
        2 done
        3 done
        check 2
      4 done
      5 done
      check 1
    0 done
    check 2
    check 4
    check 5
    check 3

```

```

dfs(11)
  check 4
  dfs(12)
    dfs(9)
      check 11
      dfs(10)
        check 12
        10 done
        9 done
        12 done
      11 done
      check 9
      check 12
      check 10

```

```

dfs(6)
  check 9
  check 4
  dfs(8)
    check 6
    8 done
    check 0
  6 done

```

```

dfs(7)
  check 6
  check 9
  7 done
  check 8

```


Strong components: complessità

- Teorema: Con l'algoritmo di Kosaraju-Sharir, si calcolano le strong component di un grafo orientato in tempo proporzionale a $V + E$.
- Dimostrazione:
- Correttezza:
 - Lemma: sia C una componente fortemente connessa di G e sia v non appartenente a C . Se w è un vertice di C tale che $w \rightarrow v$, allora v appare prima di ogni nodo di C nel reverse postorder di G_R ;

Strong components: complessità

- Correttezza: Per induzione si suppongono identificate dall'algoritmo le prime i componenti fortemente connesse (e quindi marcati tutti e soli i vertici di quelle componenti). Sia s il primo vertice non marcato nella sequenza restituita dalla fase 1. La chiamata $\text{dfs}(G, s)$ visita tutti e soli i nodi della strong component che contiene s , perché: i vertici nelle prime i componenti non vengono visitati; i vertici nella componente $i + 1$ vengono visitati perché non sono già marcati e sono raggiungibili da s ; i vertici nelle componenti successive a $i + 1$ non vengono visitati: se, per assurdo, un vertice v della componente $i + 2$ fosse visitato allora ci sarebbe un arco da un vertice della componente $i + 1$ a v , ma in virtù del lemma, tale vertice v dovrebbe apparire prima di ogni vertice della componente $i + 1$ (e quindi anche di s) nel reverse postorder.
- Complessità: sono due passate di DFS.

Implementazione

```
public class KosarajuSharirSCC {
    private boolean[] marked;
    private int[] id;
    private int count;

    public KosarajuSharirSCC(Digraph G) {
        DepthFirstOrder dfs = new DepthFirstOrder(G.reverse());
        marked = new boolean[G.V()];
        id = new int[G.V()];
        for (int v : dfs.reversePost()) {
            if (!marked[v]) {
                dfs(G, v);
                count++;
            }
        }
    }

    private void dfs(Digraph G, int v) {
        marked[v] = true;
        id[v] = count;
        for (int w : G.adj(v))
            if (!marked[w]) dfs(G, w);
    }

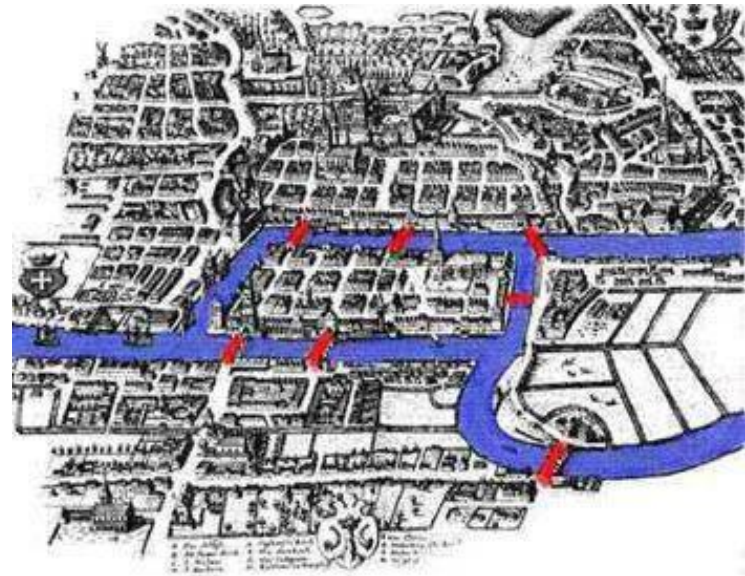
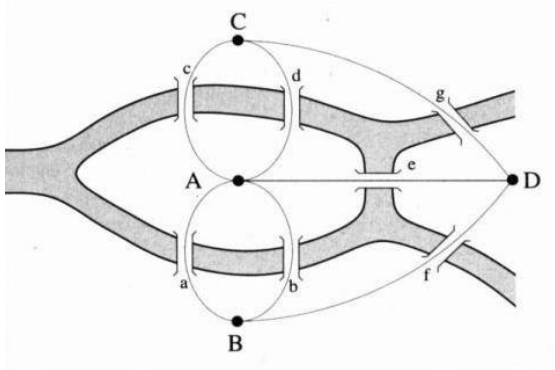
    public boolean stronglyConnected(int v, int w) {
        return id[v] == id[w];
    }
}
```



Problemi famosi sui grafi

I sette ponti di Königsberg

- La teoria dei grafi ha una data di nascita precisa: il 1736. In quella data, il matematico svizzero Leonhard Euler risolse il problema noto come i sette ponti di Königsberg.



- Ci si chiedeva se fosse possibile fare una passeggiata in città, che partisse e arrivasse allo stesso punto, in modo da attraversare tutti i ponti esattamente una volta. Un ciclo Euleriano è un ciclo che contiene tutti gli archi del grafo ciascuno una sola volta.

Ciclo euleriano in un grafo non orientato

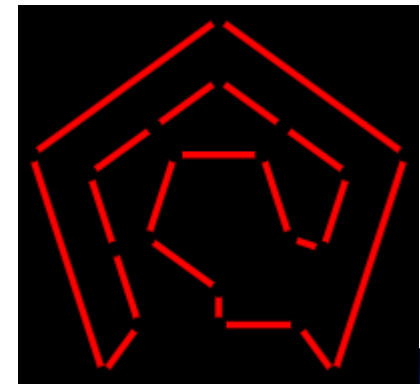
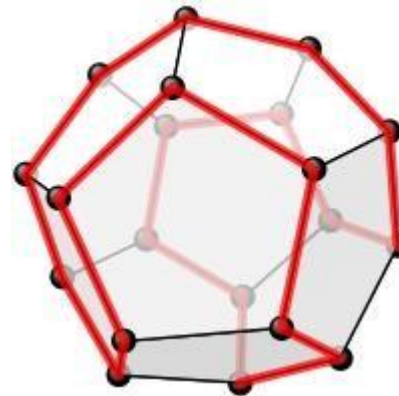
- **TEOREMA:** Un grafo ha un ciclo euleriano se e solo se G è connesso e ogni vertice ha grado pari.
- **DIMOSTRAZIONE**
- **Necessità.** Se esiste un ciclo Euleriano, allora il grafo è connesso e il cammino visita tutti i nodi. Ogni volta che raggiungiamo un nodo v , lo dobbiamo lasciare con un altro lato, quindi il suo grado è pari.
- **Sufficienza.** Supponiamo che G sia connesso e che tutti i vertici abbiano grado pari, ovvero ogni nodo deve avere almeno grado 2. In G deve esistere un ciclo F (se così non fosse il grafo sarebbe un albero, in cui sono presenti vertici di grado 1). Consideriamo il sottografo H ottenuto rimuovendo da G i lati in F . I nodi in H che non sono isolati hanno ancora grado pari. Allora da ogni componente connessa di H posso ottenere un ciclo F_i . Continuo finché non riduco tutto il grafo a nodi isolati. Per costruire il ciclo euleriano, partiamo da F ; ogni volta che incontriamo un nodo che sta in un altro ciclo F_i , aggiungiamo tutto il ciclo F_i al nostro cammino. Alla fine, si ottiene un ciclo euleriano.

Cicli euleriani in grafi orientati

- Si può provare un risultato analogo per i grafi orientati?
- Suggerimento: Si prova che un grafo orientato G ha un ciclo orientato Euleriano se e solo se G è connesso e ogni vertice ha indregree pari al suo outdegree

Il gioco del dodecaedro

- Un circuito euleriano attraversa una ed una sola volta ogni arco di un grafo. Il problema duale (solo apparentemente simile, ma molto più difficile ovvero lo stabilire se un certo grafo ammette un ciclo che passa una ed una sola volta per ogni vertice, ha anch'esso una data di nascita precisa: il 1859, quando sir William Rowan Hamilton, inventore dell'icosian game, propose il seguente rompicapo:
- Problema del viaggiatore sul dodecaedro. Ai venti vertici di un dodecaedro regolare sono assegnati i nomi di altrettante città famose. E' possibile per un viaggiatore che parta da un dato vertice-città, visitare una e una sola volta tutte le città, trasferendosi lungo gli spigoli del dodecaedro, e facendo alla fine ritorno alla città di partenza? Il problema si traduce in termini di grafi (cosa che lo stesso Hamilton suggerì) nel modo seguente: esiste un ciclo semplice nel grafo del dodecaedro che passa (una ed una sola volta) per tutti i vertici del grafo? (la risposta è sì).



Ciclo hamiltoniano

- Un grafo in cui esiste un ciclo hamiltoniano si chiama grafo hamiltoniano.
- Mentre è facile implementare un programma efficiente (qualsiasi cosa questo ragionevolmente significhi) che stabilisce se un grafo è euleriano, in generale decidere se un dato grafo ammetta un ciclo (o anche un cammino) hamiltoniano è un problema molto più difficile. E' stato dimostrato che il problema di stabilire se un grafo è hamiltoniano è NP-completo.
- Questa questione ha da sempre attirato l'interesse di diversi studiosi, e sono stati ottenuti vari risultati i quali assicurano che, sotto condizioni di solito abbastanza specifiche, certi grafi sono (o non sono) hamiltoniani.